Introduction to XML

XPath

Lesson Objectives

- At the end of this lecture, you should be able to
  - Describe the application of XML
  - Identify Valid XML tags
  - Describe the tree structure of a XML document
  - Retrieve data from XML document using XQuery

What is XML?

- XML stands for **eXtensible Markup Language.**
- XML tags provide information about a document.
- HTML tags tell a browser how to display the document.
- While HTML uses tags for formatting (e.g., “italic”), XML uses tags for semantics (e.g., “this is an address”).

Why use XML?

- XML documents are used to transfer data from one place to another often over the Internet.
- This note is a note to Tove from Jani, stored as XML:

```
<note>
   <to>Tove</to>
   <from>Jani</from>
   <heading>Reminder</heading>
   <body>Don't forget me this weekend!</body>
</note>
```

Output

Note

To: Tove

From: Jani

Reminder

Don't forget me this weekend!

Advantages of XML

- XML is text (Unicode) based.
  - **Takes up less space.**
  - **Can be transmitted efficiently.**
- One XML document can be displayed differently in different media.
  - **Html, video, CD, DVD,**
  - **You only have to change the XML document in order to change all the rest.**
- XML documents can be modularized. Parts can be reused.

Example: XML Document

```
<bookstore>
<book category="COOKING">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2006</year>
  <price>35.00</price>
</book>
<book category="CHILDREN">
……
```

XML Rules :Tags

- Tags are enclosed in angle brackets.
- Tags come in pairs with start-tags and end-tags.
- Tags must be properly nested.
  - **`<name><email>…</name></email>` is not allowed.**
  - **`<name><email>…</email></name>` is allowed.**
- Tags that do not have end-tags must be terminated by a ‘/’.
  -
 is an html example.

XML Rules :Tags continued

- Tags are case sensitive.
  - **<address> is not the same as <Address>**
- XML in any combination of cases is not allowed as part of a tag.
- Tags may not contain ‘<‘ or ‘&’.
- Tags follow Java naming conventions, except that a single colon and other characters are allowed. They must begin with a letter and may not contain white space.
- Documents must have a single root tag that begins the document.

Are these valid Tags?

- <Name>
- <name> </NAME>
- <Address&>
- < student name>
- <20student>
- <student20>
- <ADDRESS>

Well-Formed XML

- Start the document with a **declaration**, surrounded by `<?xml … ?>` .
- Normal declaration is:

```
<?xml version = “1.0”?>
```

- Balance of document is a **root tag** surrounding nested tags.

Well-Formed Documents

- An XML document is said to be well-formed if it follows all the rules.
- An XML parser is used to check that all the rules have been obeyed.
- Recent browsers such as Internet Explorer 5 and Netscape 7 come with XML parsers.
- Parsers are also available for free download over the Internet. One is Xerces, from the Apache open-source project.
- Java 1.4 also supports an open-source parser

XML Example Revisited

```
<?xml version="1.0"/>
<address>
        <name>Alice Lee</name>
        <email>alee@aol.com</email>
        <phone>212-346-1234</phone>
        <birthday>1985-03-22</birthday>
</address>
```

- Markup for the data aids understanding of its purpose.
- A flat text file is not nearly so clear.
  - `Alice Lee`
  - `alee@aol.com`
  - `212-346-1234`
  - `1985-03-22`
- The last line looks like a date, but what is it for?

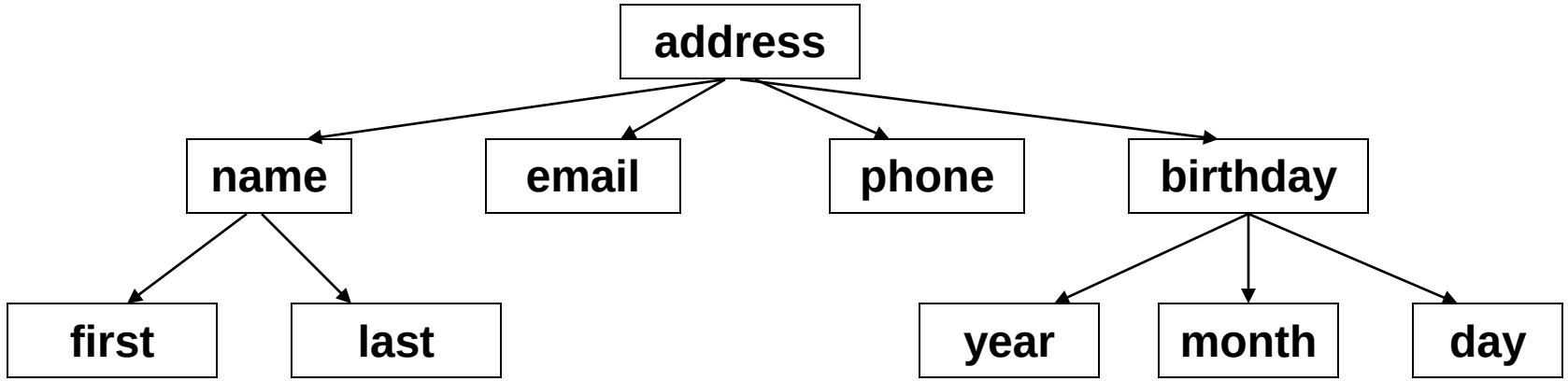
Expanded Example

```
<?xml version = “1.0” ?>
<address>
      <name>
            <first>Alice</first>
            <last>Lee</last>
  </name>
  <email>alee@aol.com</email>
  <phone>123-45-6789</phone>
  <birthday>
            <year>1983</year>
            <month>07</month>
           <day>15</day>
     </birthday>
</address>
```

XML Trees

- An XML document has a single root node.
- The tree is a general ordered tree.
  - **A parent node may have any number of children.**
  - **Child nodes are ordered, and may have siblings**.
- Preorder traversals are usually used for getting information out of the tree.

XML File is a Tree



Validity of XML Document

- A well-formed document has a tree structure and obeys all the XML rules.
- A particular application may add more rules in either a DTD (document type definition) or in a schema.

Document Type Definitions (DTD)

- A DTD describes the tree structure of a document and something about its data.
- There are two data types, PCDATA and CDATA.
  - **PCDATA is parsed character data.**
  - **CDATA is character data, not usually parsed**.
- A DTD determines how many times a node may appear, and how child nodes are ordered.

DTD for address Example

```
<!ELEMENT address (name, email, phone, birthday)>
<!ELEMENT name (first, last)>
<!ELEMENT first (#PCDATA)>
<!ELEMENT last (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
<!ELEMENT birthday (year, month, day)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT month (#PCDATA)>
<!ELEMENT day (#PCDATA)>
```

Untyped and Typed XML data type

For MS SQL Server:

Untyped XML Column in Table:

```
CREATE TABLE AdminDocs (
    id int  primary key,
    xDoc XML not null
    )
```

Typed XML Column in Table:

```
CREATE TABLE AdminDocs (
    id int primary key,
    xDoc XML (CONTENT myCollection)
    )
```

Schema collection

Untyped and Typed XML data type

Inserting Data: Example

```
INSERT INTO AdminDocs VALUES (1,
'<book >
  <title>Writing Secure Code</title>
  <author>
    <first-name>Michael</first-name>
    <last-name>Howard</last-name>
  </author>
  <author>
    <first-name>David</first-name>
    <last-name>LeBlanc</last-name>
  </author>
  <price>39.99</price>
</book>')
```

How to Query XML data?

- Main two ways to extract data

1. **Path expressions**

   great if you just want to copy certain elements and attributes as it is

2. **XQuery expressions**

   XQuery extends XPath to a query language that has power similar to SQL.

Xpath Expressions

- XPath uses path expressions to select nodes or node-sets in an XML document. The node is selected by following a path or steps.
- XPath can be used to navigate through elements and attributes in an XML document.
- In XPath, there are seven kinds of nodes:

element, attribute, text, namespace, processing-instruction, comment, and document nodes.

- XML documents are treated as trees of nodes. **The topmost element of the tree is called the root element.**

Path Descriptors

- Simple path descriptors are sequences of tags separated by slashes (/).
- If the descriptor begins with /, **then the path starts at the root and has those tags, in order.**

  /bookstore/book

- If the descriptor begins with //, **then the path can start anywhere.**

  //book

Value of a Path Descriptor

- Each path descriptor, applied to a document, has a value that is a sequence of elements.
- An element is an atomic value or a node.
- A node is matching tags and everything in between.

Example

```
<book>
  <title>Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
```

nodes in the example XML document :

- **root element node:**

```
<bookstore>
```

- **element node:**

```
<author>J K. Rowling</author>
```

- **attribute node :**

```
lang="en"
```

- **Atomic values:**
  - Atomic values are nodes with no children or parent.
  - Example of atomic values:

```
J K. Rowling ,"en"
```

- **Items**
  - Items are atomic values or nodes.

Example

```
<bookstore>
<book>
  <title>Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
</bookstore>
```

Relationship of Nodes

- **Parent:**

Each element and attribute has one parent.

Example: the book element is the parent of the title, author, year, and price

- **Children:**

Element nodes may have zero, one or more children.

Example: the title, author, year, and price elements are all children of the book element

- **Siblings:**

Nodes that have the same parent.

Example: the title, author, year, and price elements are all siblings

Relationship of Nodes Contd…

Ancestors:

- A node's parent, parent's parent, etc.
- Example: the ancestors of the title element are the book element and the bookstore element.

Descendants:

- A node's children, children's children, etc.
- Example: descendants of the bookstore element are the book, title, author, year, and price elements.

Selecting Nodes

Expression	Description
nodename	Selects all nodes with the name "nodename"
/	Selects from the root node
//	Selects nodes in the document from the current node that match the selection no matter where they are
.	Selects the current node
..	Selects the parent of the current node
@	Selects attributes

Selecting Nodes

Path Expression	Result
bookstore	Selects all nodes with the name "bookstore"
/bookstore	Selects the root element bookstoreNote: If the path starts with a slash (/) it always represents an absolute path to an element!
bookstore/book	Selects all book elements that are children of bookstore
//book	Selects all book elements no matter where they are in the document
bookstore//book	Selects all book elements that are descendant of the bookstore element, no matter where they are under the bookstore element

Exercises

1. Display all the book elements in the bookstore.
2. Display the title elements of all books.
3. Display all the author elements are in the book store.
4. Display all the price elements in the books.

Attributes

- In XPath, we refer to attributes by prepending @ to their name.
- Attributes of a tag may appear in paths as if they were nested within that tag.

Predicates

- Predicates are used to find a specific node or a node that contains a specific value.
- Predicates are always embedded in square brackets.
- **Example:**
- **Selects all the price nodes with a price higher than 35**

```
/bookstore/book[price>35]/price
```

Attributes in selection

- For the predicate we can have the attribute.
- **Example:**

  Find out the titles of the book in WEB category

```
/bookstore/book[@category="WEB"]/title
```

Example: Predicates and Returned Elements

- Using a number in a predicate does not mean that number elements are returned.
- Example:

Display all the book elements whose price child is greater than 35

```
//book[price>35]
```

```
<book category="WEB">
  <title lang="en">XQuery Kick Start</title>
  <author>James McGovern</author>
  <author>Per Bothner</author>
  <author>Kurt Cagle</author>
  <author>James Linn</author>
  <author>Vaidyanathan Nagarajan</author>
  <year>2003</year>
  <price>49.99</price>
</book>
<book category="WEB">
  <title lang="en">Learning XML</title>
  <author>Erik T. Ray</author>
  <year>2003</year>
  <price>39.95</price>
</book>
```

All the price element whose value is greater than 35

```
//book/price[. gt 35]
```

```
<price>49.99</price>
<price>39.95</price>
```

Using Position in Predicates

- Can use a number in the predicate to indicate the position of the child
- **Example:**

The 2nd book child in the bookstore.

```
//book[2]
```

```
<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2004</year>
  <price>29.99</price>
</book>
```

The 2nd child of bookstore(regardless of its name)

```
//book/*[2]
```

```
<author>Giada De Laurentiis</author>
<author>J K. Rowling</author>
<author>James McGovern</author>
<author>Erik T. Ray</author>
```

Exercises

1. Display the title of the first book node under the bookstore element.
2. Display all the title nodes with a price higher than 35.
3. Display the text from all the title nodes.
4. Display the title of the book written by J K. Rowling.
5. Display all the book elements which were published after 2003.
6. Display all the book elements in which price is greater than to 40 and category is web .
7. Display the first book node that has the price greater than to 30.

Wild-Card *

- A star (*) in place of a tag represents any one tag. (substitute any of the tag)
- Example:

Display all the book details in which price is greater than 30

```
/*/book[price > 30]
```

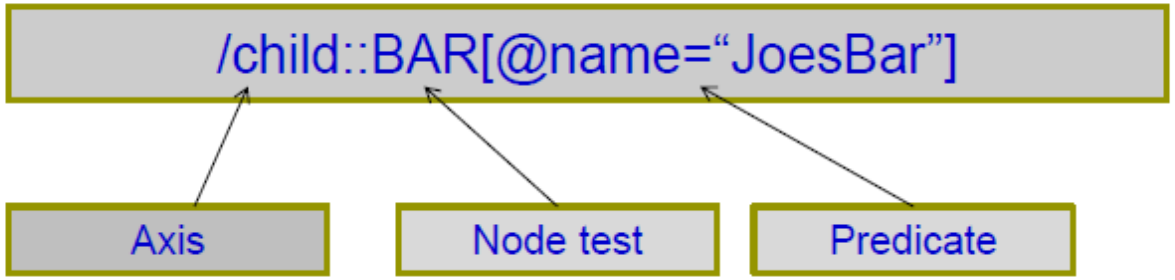
Display the titles of all the books

Method I : `/*/*/title`

Method II: `/*/book/title`

Axes

- In general, path expressions allow us to start at the root and execute steps to find a sequence of nodes at each step.
- At each step, we may follow any one of several axes.



1. The axis (Optional)
   - Direction to navigate
2. The node test
   - The node of interest by name
3. Predicate
   - The criteria used to filter nodes

Axes Continued

- @ is really shorthand for the attribute:: axis.
- **Example:**

Find the title of the books are in web category.

```
//book[@category="WEB“]/title
```

Same xpath we can write use the axis:

```
//child::book[attribute::category="WEB"]/title
```

More Axes

Some other useful axes are:

- parent:: = parent(s) of the current node(s).
- descendant-or-self:: = the current node(s) and all descendants.
- Note: // is really shorthand for this axis.
- ancestor::, ancestor-or-self, etc.
- the default axis is child:: --- go to all the children of the current set of nodes.

Exercises

1. Display all the book elements
2. Display the title elements of all books.
3. Display all the author elements are in the book store.
4. Display all the price elements in the books.

Summary

- What are XML documents?
- Why we use XML document?
- Valid Tags in XML
- Tree Structure of a XML document
- Retrieve data from XML documents

Thank you!